Master Programme in Data Science and Business Informatics

Data Mining: Advanced Topics and Applications

Alessia Delconte, Ilenia Fortuna, Giulia Mela

A.Y. 2024/2025

Contents

1 Data understanding and preparation **3**
- 1.1 Dataset . . . 3
- 1.2 Data Cleaning and Variable Selection . . . 3

2 Outlier Detection **4**
- 2.1 Comparison between Methods . . . 4
- 2.2 Qualitative Analysis of Outliers: Categorization . . . 5
- 2.3 Handling Outliers . . . 6

3 Imbalanced Learning **6**
- 3.1 Resampling Techniques . . . 7
  - 3.1.1 Undersampling . . . 7
  - 3.1.2 Oversampling . . . 7
  - 3.1.3 Combined Approach . . . 7
- 3.2 Balancing at the Algorithm Level . . . 8
- 3.3 Final Consideration . . . 8

4 Advanced Classification **8**
- 4.1 Logistic Regression . . . 9
- 4.2 Support Vector Machines . . . 9
  - 4.2.1 Linear Support Vector Classification . . . 9
  - 4.2.2 Non-Linear Support Vector Classification . . . 10
- 4.3 Neural Network . . . 10
- 4.4 Random Forests . . . 11
- 4.5 Gradient Boosting Machines . . . 12
- 4.6 Summary and Comparative Insights . . . 13

5 Advanced Regression **13**
- 5.1 LightGBM . . . 13
- 5.2 XGBoost . . . 14
- 5.3 Model Comparison . . . 14

6 Explainability **14**
- 6.1 SHAP . . . 15
- 6.2 Counterfactual Explanations . . . 15
- 6.3 LIME . . . 16
- 6.4 Conclusion . . . 16

7 Timeseries **17**
- 7.1 Exploratory Data Analysis . . . 17
  - 7.1.1 Distribution of Daily Revenues (Days 0–99) . . . 17
  - 7.1.2 Distribution of rating, rating_category, genre . . . 18
  - 7.1.3 Heatmap of Correlation . . . 18
- 7.2 Preprocessing of multiple univariate time series . . . 19
- 7.3 Anomaly Detection . . . 19
  - 7.3.1 Anomaly Detection – Local . . . 19
  - 7.3.2 Anomaly Detection – Global . . . 19
- 7.4 Motifs e Discords Analysis . . . 20

7.4.1 Intra-Series Analysis (Single Serie) 20
7.4.2 Inter-Series Analysis (Across Films) 21
7.5 Clustering 22
7.5.1 Partitional Clustering: KMeans 22
7.5.2 Feature-based clustering. 24
7.5.3 Comparison and Conclusions 25
7.5.4 Cross-Analysis Between Clustering and Anomaly Detection 25
7.6 Classification 26
7.6.1 K-Nearest Neighbors (KNN) 26
7.6.2 Shapelets 27
7.6.3 Deep Classification 28
7.6.4 Conclusion 29
7.7 Sequential Pattern Mining 30

1 Data understanding and preparation

1.1 Dataset

The IMDb dataset used in this project contains information on 149.531 forms of visual entertainment, such as movies, TV series, and more. Each piece of content is described by 32 variables that represent its essential characteristics. The following table lists the 32 variables in the dataset, with a brief description of each and its respective datatype.

Name	**Description**	**Type**
originalTitle	Original title, in the original language	object
rating	IMDB title rating class	object
startYear	Release year of a title or series start year	int64
endYear	TV Series end year	float64
runtimeMinutes	Runtime of the title, in minutes	float64
awardWins	Number of awards won	int64
numVotes	Number of votes received	int64
worstRating	Worst title rating	int64
bestRating	Best title rating	int64
totalImages	Number of images on IMDb	int64
totalVideos	Number of videos on IMDb	int64
totalCredits	Number of credits listed	int64
criticReviewsTotal	Number of critic reviews	int64
titleType	Title format (movie, series, etc.)	object
awardNominationsExcludeWins	Award nominations excluding wins	int64
canHaveEpisodes	Can the title have episodes	bool
isRatable	Can users rate this title	bool
isAdult	Whether the title is for adults (0/1)	int64
numRegions	Number of regions for this version	int64
userReviewsTotal	Number of user reviews	int64
ratingCount	Number of user ratings submitted	int64
countryOfOrigin	Country of production	object
genres	Genre(s) (e.g., drama, action)	object
castNumber	Number of cast members	int64
CompaniesNumber	Number of companies involved	int64
averageRating	Average user rating	float64
regions	Regions for this version	object
externalLinks	Number of external links on IMDb	int64
quotesTotal	Number of quotes listed	int64
writerCredits	Number of writer credits	int64
directorCredits	Number of director credits	int64
soundMixes	Sound mix specifications	object

Table 1.1: Description of the features

1.2 Data Cleaning and Variable Selection

During the data cleaning and feature selection process, several variables were removed because they were either redundant or not informative:

- **endYear** was removed because it contained over 96% missing values, making it largely incomplete and not useful for analysis.
- **worstRating** and **bestRating** were dropped since they had constant values (0 and 10, respectively) and did not provide useful information.
- **isRatable** was excluded as it contained only `True` values, offering no variability.
- **isAdult** was removed because the "Adult" category was already captured within the **genres** variable, making it redundant.
- **rating** was eliminated because it only indicated the range of possible ratings, while the variable **AverageRating** provided the actual average score given to the content.

- **ratingCount** was dropped in favor of **numVotes**, as both were highly correlated but **numVotes** had a lower standard deviation and was therefore considered more stable.

Furthermore, during this process, we also assessed the possibility of duplicate rows within the dataset. We identified and removed 10 duplicate entries, which were eliminated to streamline the dataset and reduce any potential noise or redundancy.

Another important modification involved the **genres** variable. Originally, this variable contained multiple genres per content, which added complexity and inconsistency. To simplify the dataset, we reduced the **genres** variable to four primary categories: *Comedy, Action, Drama,* and *Others*. These categories were chosen based on a predefined industry hierarchy, and each content was assigned to one of these classes according to the first listed genre. This change aimed to create a more manageable and consistent representation of genre information across the dataset.

The last modification concerns the handling of missing values in the **runtimeMinutes** column, where approximately 26.9% of the data was missing. These missing values were replaced with the average runtime calculated separately for each **titleType**, ensuring consistency with the typical characteristics of each category.

2 Outlier Detection

To analyze the presence of anomalous observations in the dataset, an outlier analysis was conducted by selecting the top 1% most anomalous cases. For this purpose, eight algorithms from different methodological families were used, following a well-established classification. These included histogram-based approaches such as *HBOS* (Histogram-Based Outlier Score), statistical methods like *Likelihood*, distance-based methods such as *KNN* (k-Nearest Neighbors), and density-based approaches like *LOF* (Local Outlier Factor). Additionally, more specialized methods were applied, including *CBLOF*, which combines clustering and density, *ABOD* (Angle-Based Outlier Detection), particularly suited for high-dimensional spaces, ensemble-based approaches like *LODA*, and model-based methods such as *Isolation Forest*. The analysis was conducted exclusively on continuous numerical variables, which were first standardized using z-score normalization to ensure consistency across the different metrics and scales employed by the algorithms.

2.1 Comparison between Methods

After applying each algorithm individually and visualizing the results in a two-dimensional space via PCA reduction, a systematic comparison of the detected outliers was performed by evaluating the overlap among the methods. Two aggregation strategies were explored: outliers detected by all eight methods, amounting to 30 cases; and those identified by at least five methods, amounting to 869 cases. This analysis helped to understand the level of agreement between the different methods and provided a solid foundation for deciding how to proceed in subsequent anomaly management phases.

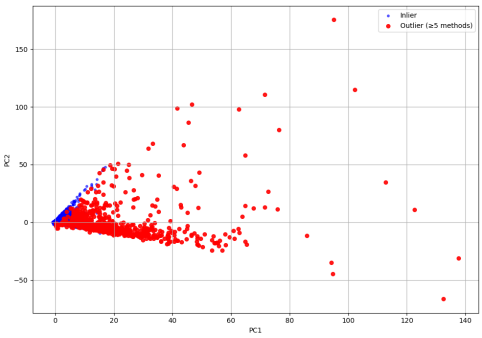


Figure 2.1: Outliers identified by at least 5 methods

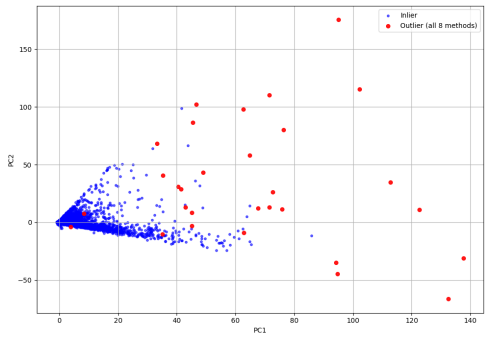


Figure 2.2: Outliers identified by all methods

Then, we wanted to understand which records were actually considered the most anomalous based on the scores obtained from all eight methods. For this purpose, we calculated the average outlier score for each of the 30 titles detected by all algorithms. This allowed us to rank them according to the overall intensity of their anomaly, as shown in the resulting chart (Figure 2.3).

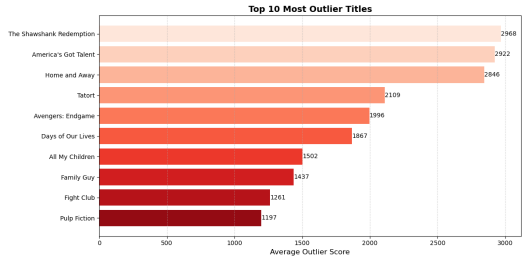


Figure 2.3: Top 10 most outlier titles

2.2 Qualitative Analysis of Outliers: Categorization

Following the identification of key outliers — specifically those detected by at least five outlier detection methods — a qualitative analysis was conducted to understand the nature of the anomalies, i.e., why certain titles were classified as outliers. For this purpose, a classification was devised based on high thresholds calculated at the 95th percentile for several key numerical variables, used as benchmarks to identify exceptionally high values. The outliers were then categorized into four groups: "Successful," which includes records with a very high number of votes and awards; "Production Focused," comprising records with significant production involvement characterized by a large cast, numerous credits, and many companies; "Structural," identifying titles that are long, distributed across many regions, or have extensive multimedia content; and finally, "Others," which collects anomalous titles not fitting the previous categories.

The resulting chart (Figure 2.4) provides a visual overview of the identified groups and supports the idea that the classification was meaningful. Although the separation between categories is not always sharp, each group displays distinct trends in specific variables, suggesting that the outliers do not represent a single type of anomaly.
For example, successful titles stand out not only for their high number of votes and awards — as expected — but also for elevated values across several other dimensions, such as credits, runtime, and production complexity, which is consistent with what we typically see in real-world successful movie productions. This analysis thus helped uncover hidden patterns within the outliers, transforming what might initially appear as isolated exceptions into interpretable and structured phenomena.

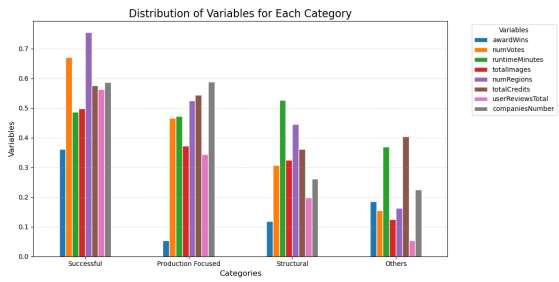


Figure 2.4: Distribution of variables for each category

2.3 Handling Outliers

Based on the initial analysis, it was decided not to remove outliers from the dataset, as many of them correspond to highly successful or culturally significant content, such as well-known or award-winning movies or series. These instances are not considered errors or noise but rather valuable data points that can enrich subsequent analyses.

To reduce the influence of these extreme values while retaining their informative value, logarithmic and square root transformations were applied to several numerical variables exhibiting skewed distributions. These transformations helped reduce the dispersion and bring the data closer to a normal distribution. To validate the effectiveness of this approach, some of the previously used outlier detection methods were reapplied to the transformed dataset. As shown in the PCA visualization using KNN (figure 2.5), the distribution of data appears more compact and the identified outliers are located closer to the main data cluster. This confirms that the transformations had a positive effect in reducing the extremity of the original values.

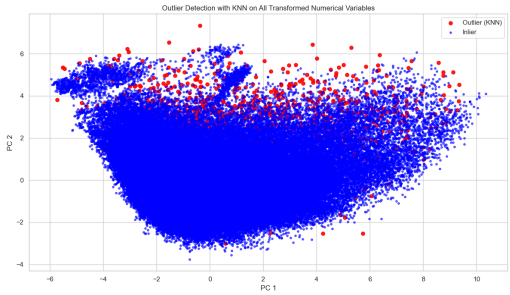


Figure 2.5: Reapplied KNN (1%) on PCA of transformed variables.

3 Imbalanced Learning

To address the issue of imbalanced learning, we defined a binary classification task aimed at predicting whether a title has won at least one award. The target variable used was derived from the ‘awardWins‘ variable. It indicates whether a content received any award, assigning a value of 1 to awarded titles and 0 otherwise. The class distribution is highly imbalanced: only about 8% of the records belong to the positive class (award winners), while the remaining 92% have not received any awards.

Our initial approach involved training a Decision Tree classifier on the original, imbalanced dataset, with hyperparameter tuning performed through randomized search and 5-fold cross-validation, optimizing for the F1-score. Although the model reached high overall accuracy 94%, it struggled to correctly classify positive samples, heavily favoring the majority class.

Class	Precision	Recall	F1-score
0	0.95	0.98	0.97
1	0.68	0.42	0.52
Accuracy		0.94	
Macro avg	0.82	0.70	0.74
Weighted avg	0.93	0.94	0.93

Table 3.1: Classification report (Baseline)

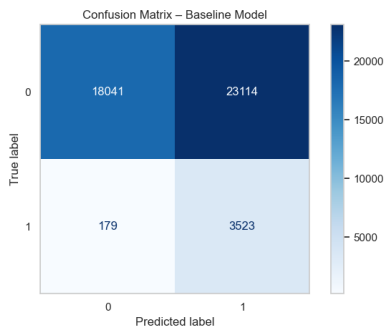


Figure 3.1: Confusion matrix

These results clearly show that the model had a strong bias toward the majority class. While it achieved high precision on the minority class (0.68), it struggled to identify most award-winning titles, as indicated by the low recall (0.42). This imbalance resulted in a poor F1-score (0.52), meaning the model was not effective at retrieving true positive cases. Because of this unbalanced

performance, we decided to apply resampling techniques to help the model learn a more useful and balanced decision boundary, especially to improve its recall while maintaining acceptable precision.

3.1 Resampling Techniques

So, we applied a variety of resampling strategies, including both **undersampling** and **oversampling** techniques. After testing several approaches, we selected the most effective methods from each category based on their performance in detecting the minority class.

3.1.1 Undersampling

At first we experimented with several undersampling techniques, including RandomUnderSampler, ClusterCentroids, and Edited Nearest Neighbours (ENN). Most undersampling methods led to unstable or unsatisfactory results, either drastically reducing recall or discarding too much information. **ENN** proved to be the most effective, especially after testing several values of k; the best results were obtained with k=5, which offered a good trade-off between removing noisy majority samples and preserving representative ones. With this approach, the model identified 2,234 true positives and only 1,468 false negatives, significantly reducing the number of missed award-winning titles compared to the baseline. False positives also dropped to 2,203, greatly improving the model's reliability.

Class	Precision	Recall	F1-score
0	0.964	0.946	0.955
1	0.503	0.603	0.549
Accuracy		0.918	
Macro avg	0.734	0.775	0.752
Weighted avg	0.926	0.918	0.921

Table 3.2: Classification report (ENN)

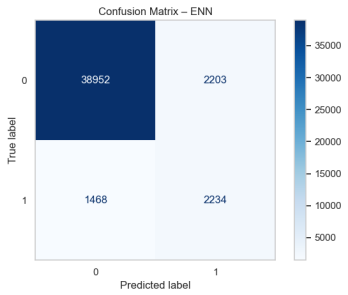


Figure 3.2: Confusion matrix

3.1.2 Oversampling

We then moved on to oversampling techniques, testing methods such as RandomOverSampler, SMOTE, and ADASYN. While several approaches were evaluated, **ADASYN** proved to be the most effective in improving minority class representation. After experimenting with different values for the number of neighbors, we found that setting k=7 led to the best results. With this configuration, the model identified 1,954 true positives and 1,748 false negatives, while maintaining a relatively low number of false positives (2,125).

Class	Precision	Recall	F1-score
0	0.957	0.948	0.953
1	0.479	0.528	0.502
Accuracy		0.914	
Macro avg	0.718	0.738	0.727
Weighted avg	0.918	0.914	0.916

Table 3.3: Classification report (ADASYN)

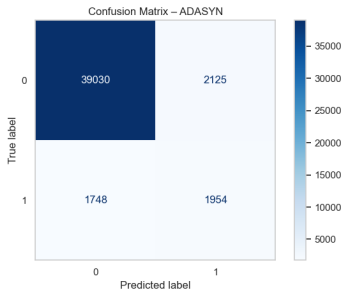


Figure 3.3: Confusion matrix

3.1.3 Combined Approach

We also evaluated a combined approach, where ADASYN was applied first to generate synthetic minority instances, followed by ENN to remove potentially ambiguous or overlapping samples. This combination aimed to exploit the strengths of both methods: enhanced coverage of the

minority class and post-hoc noise reduction. As expected, this setup significantly boosted the recall, allowing the model to correctly identify 2,893 award-winning records (the highest among all models) and misclassifying only 809. However, this came at the cost of precision: the model generated 6,513 false positives, which represents a substantial increase in noisy predictions. Consequently, although this pipeline achieved the best recall 78.1%, its F1-score was considerably lower than that of ENN alone.

Class	**Precision**	**Recall**	**F1-score**
0	0.977	0.842	0.904
1	0.308	0.781	0.441
Accuracy		0.837	
Macro avg	0.642	0.812	0.673
Weighted avg	0.922	0.837	0.866

Table 3.4: Classification report (ADASYN + ENN)

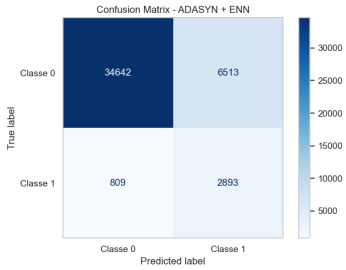


Figure 3.4: Confusion matrix

3.2 Balancing at the Algorithm Level

Beyond resampling, we also explored algorithm-level strategies to address class imbalance. This included adjusting the `class_weight` parameter of the Decision Tree and modifying the classification threshold.
While increasing the weight of the minority class (e.g., 0:1, 1:5) improved recall (up to 0.55), it lowered precision (around 0.39), resulting in modest F1-scores (0.46). Similarly, increasing the decision threshold (e.g., to 0.9) led to high precision (0.81) but very low recall (0.18), with an F1-score of only 0.30.
Since these results were less balanced and effective compared to those obtained through resampling (such as ENN or ADASYN), we chose not to include them in the final comparison, but we report them here for completeness.

3.3 Final Consideration

The experiments conducted highlighted the limitations of training on imbalanced data, especially in terms of detecting the minority class. Among all the tested methods, Edited Nearest Neighbours (ENN) offered the most balanced and robust performance. It significantly reduced false positives while still identifying a meaningful number of award-winning titles, resulting in a noticeable improvement in the F1-score compared to the baseline. This confirms ENN as a particularly effective solution when the goal is to mitigate class imbalance without overly distorting the original data distribution.

4 Advanced Classification

The objective of this section is to compare the effectiveness of several advanced classification algorithms in a multi-class context. For our experiments, we decided to use two different target variables:

- `rating_class`: a discretization of the `averageRating` variable into six ordinal categories (**very low, low, medium-low, medium, high, very high**);
- `macro_titleType`: a variable derived from the original `titleType`, initially consisting of ten distinct classes, which was regrouped into six broader categories (**tv_episode, movie, short_content, tv_series, tv_movie, digital_media**) to simplify the classification task and achieve a more balanced class distribution.

The analysis was conducted using a selection of advanced classification models, including Logistic Regression, Support Vector Machines, Neural Networks (MLP), Random Forest and LightGBM.

During the experimental phase, we observed that `rating_class` suffered from a strong class imbalance, which negatively affected model performance when left unaddressed. For this reason, we applied balancing techniques only to this target. In particular, we tested different resampling strategies and found that Edited Nearest Neighbours (ENN) with $k = 3$ produced the best results for Logistic Regression and Support Vector Machines, outperforming other values of k and alternative techniques. Conversely, for Random Forest, Multi-Layer Perceptron (MLP), LightGBM, SMOTE helped improve results, although performance remained limited due to the complexity and imbalance of the task.

For `macro_titleType`, no balancing methods were applied, since preliminary experiments showed no significant performance gains, and the class distribution was already more stable due to the category regrouping step.

4.1 Logistic Regression

Logistic Regression was used as the first classification model, and the **One-vs-Rest (OvR)** strategy was adopted to handle the multi-class setting. The hyperparameter tuning phase was carried out using **RandomizedSearchCV** with *5*-fold cross-validation and optimization of the macro `f1`-score. The parameters explored included the regularization coefficient `C` (ranging from `0.01` to `10`), the solvers `lbfgs` and `saga` (both suitable for multi-class classification), and the maximum number of iterations `max_iter` (between `500` and `2000`) to ensure convergence.The optimal hyperparameters selected were `C = 1`, `solver = 'lbfgs'`, and `max_iter = 500` for the `rating_class` task, and `C = 10`, `solver = 'saga'` and `max_iter = 1000` for the `macro_titleType` classification.

The results showed limited performance on the `rating_class` task, with low overall accuracy and difficulty distinguishing minority and intermediate classes. On the other hand, the model performed better on `macro_titleType`, especially for majority classes such as `movie` and `tv_episode`, although it still struggled with less represented categories.

Metric	**Prec.**	**Rec.**	**F1**
Accuracy		0.26	
Macro avg	0.26	0.28	0.22
Weighted avg	0.35	0.26	0.27

Table 4.1: LR – `rating_class`

Metric	**Prec.**	**Rec.**	**F1**
Accuracy		0.76	
Macro avg	0.59	0.64	0.61
Weighted avg	0.79	0.76	0.77

Table 4.2: LR – `macro_titleType`

4.2 Support Vector Machines

In this part of the analysis, we explored several models based on Support Vector Machines (SVM), testing both linear and non-linear approaches in order to determine which was more suitable for our data.

4.2.1 Linear Support Vector Classification

Although we performed PCA visualizations that revealed significant class overlap, the linear model was still tested as a baseline due to its fast training time and lower risk of overfitting. Hyperparameter tuning was conducted on C (ranging from 0.01 to 100) and `max_iter` (1000 to 5000). The best configuration for `rating_class` was $C = 0.01$ and `max_iter` = 1000, while for `macro_titleType` it was $C = 1$ and `max_iter` = 3000.

In the case of `rating_class`, performance was limited (26% accuracy, low macro F1-score), with better results only for extreme classes. Conversely, for `macro_titleType`, the model achieved 78% accuracy, correctly classifying the most frequent classes, although struggling with the less represented ones.

Accuracy		0.26	
Macro avg	0.26	0.27	0.20
Weighted avg	0.35	0.26	0.27

Table 4.3: LinearSVC – `rating_class`

Accuracy		0.78	
Macro avg	0.58	0.63	0.59
Weighted avg	0.77	0.78	0.77

Table 4.4: LinearSVC – `macro_titleType`

4.2.2 Non-Linear Support Vector Classification

As expected, the non-linear approach yielded better results compared to the linear one, confirming that the complexity of the problem requires models capable of learning non-linear decision boundaries. For this reason, we tested the `SVC` classifier using three different kernel functions: **linear**, **RBF (Radial Basis Function)**, and **polynomial**, comparing their effectiveness across both classification tasks.

Hyperparameter tuning was performed using `GridSearchCV`, optimizing for the macro F1-score. For the regularization parameter C, the values 0.1, 1, and 10 were tested across all kernels. For the RBF and polynomial kernels, the `gamma` parameter was also included, using both `scale` and 0.01. Finally, for the polynomial kernel, the `degree` parameter was tested with values 2 and 3. This configuration allowed us to compare more rigid models with more flexible ones and to evaluate how model complexity affects performance.

The **RBF kernel** achieved the best performance in both the `rating_class` and `macro_titleType` tasks, ensuring a modest balance between precision and recall, even for underrepresented classes, despite a modest overall accuracy.

Metric	Prec.	Rec.	F1
Accuracy		0.32	
Macro avg	0.32	0.32	0.27
Weighted avg	0.41	0.32	0.34

Table 4.5: SVC RBF – `rating_class`

Metric	Prec.	Rec.	F1
Accuracy		0.58	
Macro avg	0.46	0.49	0.44
Weighted avg	0.74	0.58	0.63

Table 4.6: SVC RBF – `macro_titleType`

4.3 Neural Network

An initial experiment using the `MLPClassifier` from *scikit-learn* showed limited performance in handling the complexity of multiclass classification. To better address the task, two customized neural networks were developed with *Keras* and *Scikeras*, with hyperparameter tuning performed via **RandomizedSearchCV**. Both models follow a *feed-forward MLP* architecture with two hidden layers using ReLU activation and a softmax output layer, trained with the Adam optimizer and regularized through *early stopping* (patience = 10), learning rate reduction (after 5 stagnant epochs), and a 20% validation split.

For the `rating_class` task, the best-performing configuration used 256 and 64 neurons in the two hidden layers, a learning rate of 0.001, batch size of 32, and a maximum of 100 epochs. It achieved 34.2% accuracy and a macro F1-score of 0.27, with good results on majority classes but poor performance on minority ones.

In contrast, for `macro_titleType`, a smaller and moderately regularized network was used, with

dropout values between 0.2 and 0.3 and L2 regularization set to 0.001, trained for 60 epochs. Despite its simpler architecture, the model reached 86% accuracy and a macro F1-score of 0.76, performing well on the most frequent classes and struggling with the underrepresented ones.

Class	Prec.	Rec.	F1
very low	0.00	0.00	0.00
low	0.15	0.47	0.23
medium-low	0.25	0.32	0.28
medium	0.36	0.21	0.27
high	0.51	0.35	0.42
very high	0.36	0.51	0.42
Accuracy		0.34	
Macro avg	0.27	0.31	0.27
Weighted avg	0.37	0.34	0.34

Table 4.7: **MLP** – `rating_class`

Class	Prec.	Rec.	F1
digital_media	0.45	0.67	0.54
movie	0.93	0.79	0.85
short_content	0.85	0.95	0.90
tv_episode	0.99	0.91	0.95
tv_movie	0.33	0.56	0.41
tv_series	0.87	0.97	0.92
Accuracy		0.86	
Macro avg	0.74	0.81	0.76
Weighted avg	0.89	0.86	0.87

Table 4.8: **MLP** – `macro_titleType`

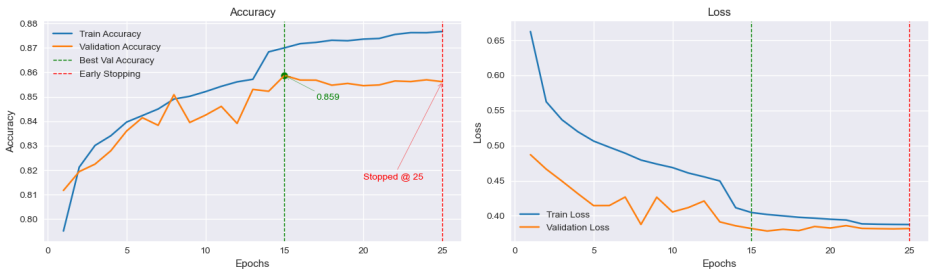


Figure 4.1: Learning Curves: Accuracy and Loss over Epochs for **macro_titleType**

4.4 Random Forests

To continue our analysis with an ensemble approach, we applied the Random Forest model, optimizing its performance through `RandomizedSearchCV` with 5-fold cross-validation.
For the classification of the **rating_class** variable, a broad hyperparameter space was explored: **n_estimators** = {100, 200}, **criterion** = {gini, entropy}, **max_depth** = {10, 20, None}, **min_samples_split** = {2, 5}, and **min_samples_leaf** = {1, 2}. The best-performing configuration was: **n_estimators = 200**, **criterion = 'entropy'**, **max_depth = 20**, **min_samples_split = 2**, **min_samples_leaf = 1**.

For the **macro_titleType** task we used the same **n_estimators**, **criterion** and **max_depth** but had a **min_samples_split** = {5, 10} and a **min_samples_leaf** = {2, 4}. The optimal model was selected with the following configuration: **criterion = 'entropy'**, **max_depth = 20**, **min_samples_split = 5**, **min_samples_leaf = 2**.
Regarding the `rating_class` variable, the model showed good performance on the most represented classes, such as `high` and `very high`, while performance dropped significantly for minority classes, particularly `low` and `very low`.

In the case of `macro_titleType`, the model achieved excellent performance (92% accuracy, macro F1-score = 0.82), even on less frequent classes. This initially raised concerns about potential overfitting; however, further tests with simpler configurations and the analysis of learning curves confirmed the model's reliability. The high performance is likely also influenced by the predictive power of certain features, such as `runtimeMinutes`, even though features too strongly correlated with the target were excluded prior to training.

Class	Precision	Recall	F1-score
very low	0.22	0.22	0.22
low	0.21	0.23	0.22
medium-low	0.32	0.30	0.31
medium	0.40	0.38	0.39
high	0.51	0.53	0.52
very high	0.47	0.48	0.47
Accuracy		0.43	
Macro avg	0.36	0.36	0.36
Weighted avg	0.43	0.43	0.43

Table 4.9: Random Forest– `rating_class`

Class	Precision	Recall	F1-score
digital media	0.74	0.61	0.67
movie	0.90	0.93	0.91
short content	0.90	0.96	0.93
tv episode	0.98	0.96	0.97
tv movie	0.56	0.46	0.51
tv series	0.91	0.99	0.95
Accuracy		0.92	
Macro avg	0.83	0.82	0.82
Weighted avg	0.91	0.92	0.91

Table 4.10: Random Forest – `macro_titleType`

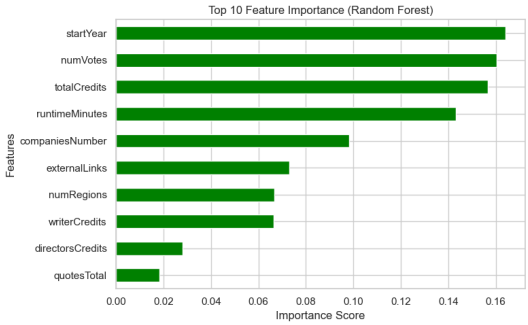


Figure 4.2: Top 10 most important features for `rating_class`

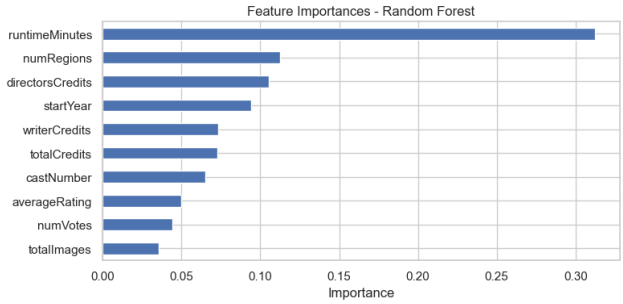


Figure 4.3: Top 10 most important features for `macro_titleType`

4.5 Gradient Boosting Machines

Among the various Gradient Boosting Machine implementations, we selected `LightGBM` due to its faster training times and better scalability, especially in multiclass and imbalanced classification scenarios. Hyperparameter tuning was carried out using `RandomizedSearchCV` with 3-fold cross-validation and macro F1-score as the evaluation metric. The search space included values for `n_estimators` (100, 200, 300), `learning_rate` (0.01, 0.05, 0.1), `max_depth` (3, 5, 7), `num_leaves` (15, 31, 63), and the subsampling parameter `subsample` (0.7, 0.8, 1.0), aiming to balance model complexity and generalization.

Since increasing the number of base estimators often improves performance up to a certain point, we found that **200 estimators offered the best trade-off** between accuracy and training time—outperforming 100 without suffering from the diminishing returns observed beyond that. On the `rating_class` task, LightGBM achieved an accuracy of 40% and a macro F1-score of 0.32, performing well on frequent classes (*high*, *very high*), while struggling with minority ones such as *very low*. The best configuration included `200 estimators`, `learning_rate = 0.05`, `max_depth = 7`, `num_leaves = 31`, `subsample = 0.80`, and `colsample_bytree = 0.70`.

For the `macro_titleType` task, performance improved substantially, with an accuracy of 89% and a macro F1-score of 0.82. The model classified major classes (*tv episode*, *tv series*, *short content*) with high precision, while still yielding acceptable results on smaller classes (*tv_movie*, *digital_media*). Interestingly, the same hyperparameter configuration proved optimal for this task as well, despite having been obtained through an independent tuning process, highlighting LightGBM's robustness across different classification settings.

Class	Prec.	Rec.	F1
very low	0.14	0.22	0.17
low	0.23	0.24	0.24
medium-low	0.33	0.22	0.26
medium	0.41	0.27	0.33
high	0.46	0.58	0.51
very high	0.39	0.42	0.41
Accuracy		0.40	
Macro avg	0.33	0.33	0.32
Weighted avg	0.39	0.40	0.39

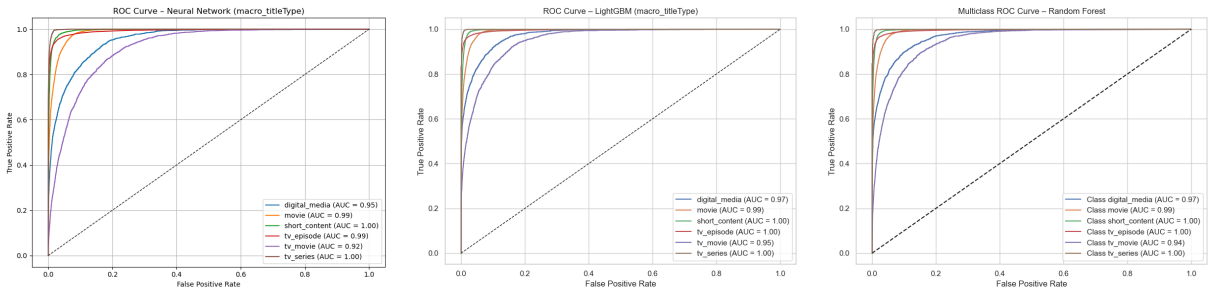
Table 4.11: **LightGBM – `rating_class`**

Class	Prec.	Rec.	F1
digital_media	0.63	0.72	0.67
movie	0.95	0.83	0.89
short_content	0.88	0.96	0.92
tv_episode	0.99	0.93	0.96
tv_movie	0.40	0.70	0.51
tv_series	0.92	0.98	0.95
Accuracy		0.89	
Macro avg	0.80	0.85	0.82
Weighted avg	0.92	0.89	0.90

Table 4.12: **LightGBM – `macro_titleType`**

4.6 Summary and Comparative Insights

The classification of the `rating_class` target proved difficult due to strong class imbalance and the intrinsic complexity of the task. All models showed limited performance, with overall accuracy below 45% and macro F1-scores around 0.30. While frequent classes such as `high` and `very_high` were predicted with moderate success, minority classes—especially `very_low`—obtained near-zero F1-scores. Among the models, LightGBM and Random Forest performed best, though their results remained unsatisfactory.
In contrast, the classification of the `macro_titleType` target was more manageable, thanks to the regrouping of categories that led to a more balanced distribution. In this setting, models such as MLP, Random Forest, and LightGBM achieved over 85% accuracy and macro F1-scores above 0.75, with strong performance on dominant classes like `tv_episode`, `movie`, and `short_content`. However, rare classes such as `tv_movie` and `digital_media` remained difficult to classify and showed inconsistent results across models.



Neural Network LightGBM Random Forest

Figure 4.4: ROC curves for the macro_titleType classification task using three different models.

Due to space constraints, we included only the ROC curves of the best-performing models on this task. Other models were tested as well, but their results are not shown here for brevity.

5 Advanced Regression

This report presents a comparative analysis of two advanced regression models — `LightGBM` and `XGBoost` — aimed at predicting the `averageRating` variable using a variety of numerical and categorical features.

5.1 LightGBM

LightGBM was the most effective model. The dataset was first cleaned by removing irrelevant textual columns, such as titles and descriptions, to reduce noise and dimensionality. Next, cat-

egorical features (for example, genre, content type, or production country) were transformed into numerical formats through encoding techniques suitable for machine learning models. Depending on the feature, this involved either one-hot encoding, which creates binary indicator variables for each category, or label encoding, which assigns integer codes to categories. After these preprocessing steps, the dataset was split into training and test sets using a standard partition (e.g., 80% training, 20% test). The model was trained on the training data and then validated on the test data to evaluate its ability to generalize to unseen examples. This workflow ensures the model's performance metrics reflect realistic predictive power.

5.2 XGBoost

To handle the computational demands of training on a large dataset, a random subset of 15,000 instances was selected for efficient model development and hyperparameter tuning. This approach ensured faster training iterations while maintaining a representative sample of the data. Numerical features were standardized and categorical features one-hot encoded, enabling the model to properly interpret the inputs. After tuning on this subset, the optimized model was retrained on the entire dataset, and performance metrics were computed on the full data to provide a comprehensive and fair evaluation. While slightly less accurate than LightGBM, XGBoost still served as a reliable baseline model.

5.3 Model Comparison

Model	**RMSE**	**MAE**	$\mathbf{R^2}$
LightGBM	1.0601	0.7758	0.3792
XGBoost	1.1147	0.8248	0.3166

Table 5.1: Summary of performance metrics for each regression model.

LightGBM achieved the best overall performance, with an RMSE of 1.0601, indicating the lowest mean squared error among the three models. The MAE of 0.7758 confirms good accuracy in terms of mean absolute error as well. Finally, an R^2 of 0.3792 suggests that approximately 38% of the variance in the target variable is explained by the model, a result superior to the other two.

XGBoost demonstrates performance with an RMSE of 1.1147 and an MAE of 0.8248, slightly worse than the other two models. The R^2 of 0.3166 confirms that the model explains about 32% of the variance in the target variable, a lower value compared to LightGBM.

In summary, all three models show good predictive capability, with LightGBM slightly outperforming in terms of accuracy and explanatory power. However, the differences are modest, and the final choice may also depend on other factors such as training speed or interpretability.

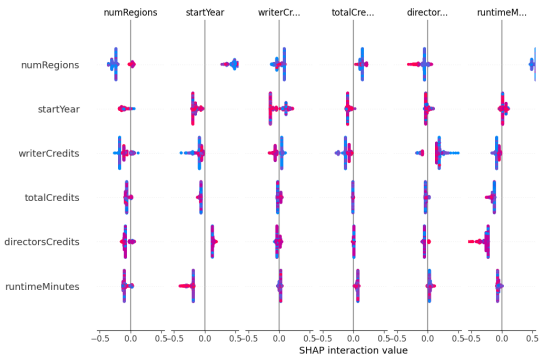
An analysis of feature importances revealed notable differences across the models. LightGBM focused on numerical and popularity-related features such as `titleType`, `startYear`, `numVotes`, and `runtimeMinutes`. XGBoost emphasized categorical distinctions, particularly specific values of `titleType` and `regions`, highlighting the relevance of content type and geographic availability.

6 Explainability

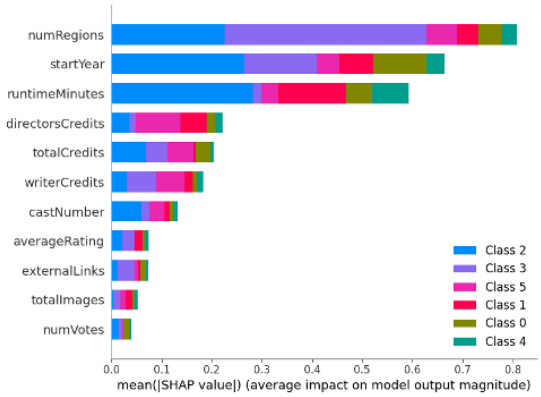
This chapter presents an explainability analysis of the Random Forest model developed for classifying audiovisual content types (target variable `titleType`). The goal is to understand which features most influence the model's predictions, both globally and locally.

6.1 SHAP

The SHAP (SHapley Additive exPlanations) method was applied to quantify the contribution of each feature to the model's predictions on the test set.
A TreeExplainer was created for the Random Forest model, which is a tree-based model. SHAP values were computed on test set to estimate how much each feature contributes to the predictions. The SHAP summary plot shows the global importance of features and their influence (positive or negative) on the model's output.
Each bar in the summary plot represents the mean absolute SHAP value (mean|SHAP|) of a feature, indicating how much that feature influences the prediction on average.
The horizontal axis shows the magnitude of the mean impact: longer bars correspond to more influential features. The bars are further divided by colors representing the different classes, showing the importance of each feature for each specific class. For example, `numRegions` has a strong impact across nearly all classes. In summary, `numRegions`, `startYear`, and `runtimeMinutes` emerged as the most important features for predicting the classes in the Random Forest model.
The SHAP interaction values plot reveals interactions between pairs of features and how these affect the prediction. The vertical and horizontal axes correspond to the two features, and each colored point represents an individual instance's interaction value (positive or negative) for that feature pair. This plot helps understand how features act together—if the combined effect differs from the sum of their individual effects. Points clustered near zero indicate little or no interaction, while spread points away from zero indicate significant interactions influencing predictions. Features such as `numRegions`, `startYear`, and `runtimeMinutes` show stronger interactions with other features, confirming their key role.



SHAP interaction values plot



Mean absolute SHAP value

Overall, SHAP interpretation enables a clear understanding of how the model makes decisions globally and locally, improving transparency and trustworthiness of the Random Forest classifier.

6.2 Counterfactual Explanations

In this section,to better understand the model's decision-making process at a local level, we employed counterfactual explanations using the DiCE framework.

For a selected subset of test instances, we generated counterfactual examples—modified versions of the original instances that the model would classify differently. For each original instance, we identified the predicted class and then selected a different target class to generate the counterfactual. The objective was to find the minimal changes required on the input features to flip the model's prediction to the desired target class.

The counterfactual generation process aims to identify minimal perturbations to the input features that result in a change of the model's predicted class. In this analysis, we focused

on cases where the predicted class of an original instance was changed from `tv_episode` to `digital_media` through small modifications of feature values.

The features most frequently adjusted to achieve the change in predicted class include `runtimeMinutes`, where increasing the runtime was often necessary to shift predictions from shorter TV episodes to longer digital media content. This indicates that the model strongly relies on duration-related information to distinguish these classes. Changes in `writerCredits` also played a significant role, suggesting that the number of credited writers helps differentiate content types, possibly reflecting production complexity. Additionally, increases in `externalLinks` were commonly observed, which may be linked to the richer metadata or greater connectivity typical of digital media content.

These targeted feature modifications highlight the decision boundaries learned by the Random Forest model. Examining such counterfactuals provides interpretable insights into the model's internal logic, aiding in validating predictions and detecting potential biases.

6.3 LIME

To gain local insights into the model's predictions, LIME (Local Interpretable Model-agnostic Explanations) was applied to the first five instances of the test set. LIME provides, for each instance, the most relevant features along with their contribution to the predicted outcome. To deepen the interpretation, the most influential features identified by LIME were selected and their distributions analyzed within the subset of instances classified into the same class as the analyzed instance. Descriptive statistics were computed to compare specific instance values with those typical of the class.

LIME explanations highlight features with strong positive or negative contributions to the predictions. For example, for instance 0, `runtimeMinutes` and `numRegions` are determinative with negative contributions, indicating that values within certain intervals reduce the probability of the assigned class.

Statistical analysis on the group of instances belonging to class 3 reveals that `runtimeMinutes` has an average value around 6.02 minutes, with instance 0 positioned close to the median, making it representative of the class. In contrast, `numRegions` has an average of about 0.70, while instance 0 records a value of 0, significantly lower than the mean, suggesting that this feature may help distinguish this instance from the rest of the group. Additionally, `startYear` shows an average near 2000, whereas instance 0 is more recent, dating to 2012, which may also serve as a discriminating factor.

This combination of local explanations from LIME and global feature analysis facilitates understanding both individual model decisions and the typical characteristics of each class. It also enables identification of atypical or borderline instances within their class.

6.4 Conclusion

The combined use of SHAP values, counterfactual explanations, and LIME provided a comprehensive understanding of the Random Forest model's decision-making process. SHAP offered a global and local perspective on feature importance, clearly identifying key variables such as `numRegions`, `startYear`, and `runtimeMinutes` that significantly influence predictions across classes. The interaction analysis further confirmed the interplay between these important features, revealing how their combined effects impact model outputs. Conversely, counterfactual explanations complemented this view by focusing on specific instances, revealing minimal feature changes necessary to alter the predicted class. These local "what-if" scenarios, especially involving features like `runtimeMinutes`, `writerCredits`, and `externalLinks`, provided actionable insights into the model's decision boundaries and helped uncover how feature variations drive classification shifts. Additionally, LIME explanations gave interpretable local approximations of the model's behavior, highlighting feature contributions for individual predictions and

allowing comparison of instance-specific values to overall class distributions. Together, these interpretability techniques enhanced model transparency by linking quantitative global importance with qualitative instance-level explanations.

7 Timeseries

7.1 Exploratory Data Analysis

The dataset `imdb_ts.csv` consists of 1,134 films, each observed over the first 100 days of box-office performance. Each row corresponds to one title and begins with the IMDb identifier (`ttXXXXXXX`), which allows us to link the data to official metadata. Following that are one hundred numeric columns labeled "0" through "99", where each represents the revenue in US dollars collected on that precise day in theaters (day 0 is opening day, day 1 the next day, and so on). These values record with precision the daily box-office trajectory: in the first days, a few blockbusters may exceed tens of millions, while at the end of their run most films drop to zero.
Alongside these temporal features, the file also contains contextual information that enriches the analysis: the average rating (on a 1–10 scale) and a categorical field (`rating_category`) that groups ratings into bands. There is also a `genre` column that assigns each film one (or more) genre(s) (for example, Drama, Comedy, or Action). These semantic attributes allow correlating revenue patterns with perceived audience quality and narrative type. Additional metadata used in later sections—such as the original title, runtime in minutes... —was retrieved by us using external sources like Mojo, based on the IMDb identifier. Although approximation techniques (such as SAX or PAA) are often employed to reduce dimensionality in time series analysis, in our case we chose to retain the full-resolution data. Given the short and fixed length of each series (exactly 100 points), approximation was deemed unnecessary and potentially detrimental to the detection of subtle but meaningful variations in revenue trends across films.

7.1.1 Distribution of Daily Revenues (Days 0–99)

Figures 7.1 and 7.2 visually support the statistics summarized in Table 7.1. The average revenue curve in Figure 7.1 clearly highlights the initial peak followed by a rapid decline and long stabilization, in line with the sharp drop in mean and median values across phases. Figure 7.2 confirms the high variability in the opening week (large spread and extreme outliers), which progressively narrows in the decay and long-tail periods. This trend reflects the decreasing standard deviation reported in the table and indicates greater regularity over time. Both figures thus reinforce the temporal segmentation adopted for the analysis.

Table 7.1: Descriptive statistics of box office revenues by temporal phase

Phase	**Mean**	**Median**	**Std. Dev.**	**Min**	**Max**
Opening (Days 0–6)	5,757,944	2,189,748	8,153,891	9,462	90,678,698
Decay (Days 7–29)	2,054,618	885,923	3,216,165	6,665	38,621,035
Long Tail (Days 30–99)	500,104	121,393	710,246	3,644	9,694,772

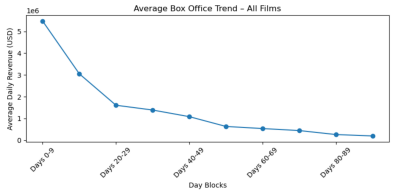


Figure 7.1: Average box office

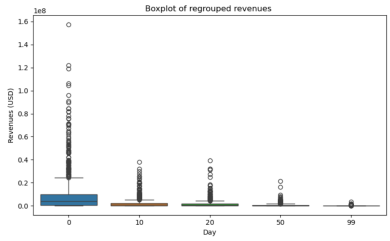


Figure 7.2: Boxplot of regrouped revenues

7.1.2 Distribution of rating, rating_category, genre

Rating Categories	Medium (387), High (377), Medium High (232), Medium Low (128), Low (10)
Top Genre Combinations	Animation, Adventure, Comedy (69), Action, Adventure, Sci-Fi (52), Comedy, Drama (45), Horror, Mystery, Thriller (40), Action, Adventure, Comedy (35)

Table 7.2: Distribution of films by rating category and most frequent genre combinations

The ratings are densely concentrated, with an average of 6.58, a median of 6.6, and a standard deviation of 0.90. Most values lie between 6.1 (25th percentile) and 7.3 (75th percentile), confirming a strong central peak. Ratings range from a minimum of 2.8 to a maximum of 8.7 across 1,134 films. The table above summarizes the distribution of titles across rating categories and the five most frequent genre combinations.

7.1.3 Heatmap of Correlation

The heatmap shows the correlation between daily box office revenues for days 0 to 99. A strong correlation between adjacent days is clearly visible along the dark diagonal band, indicating continuity in revenue trends. Regular diagonal patterns appear every 7 days, revealing a weekly seasonality. As the temporal distance increases, correlation gradually decreases but remains moderately high even after 30 days, suggesting a slow decay and medium-term memory in the revenue dynamics. This observation guided the choice of the window parameter $w = 7$ in subsequent sliding-window analyses, such as STL decomposition and the detection of recurring temporal patterns.

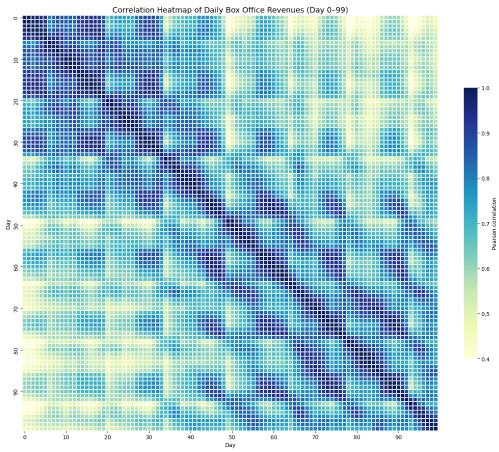


Figure 7.3: Correlation heatmap

7.2 Preprocessing of multiple univariate time series

To preprocess the 1,134 box office time series for anomaly detection and clustering, we applied a combination of normalization and decomposition techniques. Each series was first rescaled to $[0, 1]$ using Min–Max scaling, and then log-transformed to reduce extreme peaks and highlight subtler variations. We then applied STL decomposition with a weekly period ($p = 7$) to extract trend, seasonality, and residual components. The residuals, representing zero-centered fluctuations, were subsequently used as input for outlier detection.

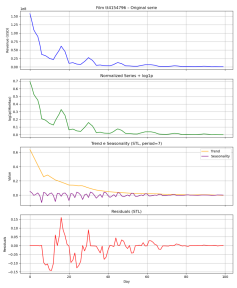


Figure 7.4: Example of STL decomposition.

7.3 Anomaly Detection

7.3.1 Anomaly Detection – Local

We began by defining two complementary anomaly-flag masks on the STL residuals. The first, `flags_any`, marks any day identified as anomalous by at least one of our three detectors (IQR, Hampel, Grubbs), prioritizing sensitivity. The second, `flags_all`, marks only those days flagged by all three methods simultaneously; we chose to continue with `flags_all` because it greatly reduces false positives and focuses on genuine data errors rather than legitimate revenue spikes.

Estimation: Applying all three tests at once over the full 100-day residual series yields the `flags_all` mask (see 7.5). Here, IQR outliers are those beyond $1.5 \times$ IQR from the quartiles, the Hampel filter flags points whose deviation from the local median (in a 7-day window) exceeds 3 MAD, and the Grubbs test ($\alpha = 1\%$) identifies statistically extreme values under a normality assumption. Requiring agreement across all three yields a robust set of confirmed outliers.
We found 163 films with at least five anomalies. Ranking these by descending count, the top five titles each exhibited seven confirmed outliers: `tt2294629` (Frozen), `tt3832914` (War Room), `tt4364194` (The Peanut Butter Falcon), `tt5027774` (Three Billboards Outside Ebbing, Missouri), and `tt6359956` (A Bad Moms Christmas).

Incremental prediction: To simulate real-time monitoring, we reran each detector cumulatively: after an initial 14-day fitting window, for each day $i \geq 14$ we applied the tests on residuals from day 0 through i, recording only whether day i itself was flagged. This yielded per-method "live" prediction masks (see 7.6). Interestingly, some films (e.g. `tt4364194` (The Peanut Butter Falcon) and `tt2294629` (Frozen)) produced new anomalies missed by the estimation approach, while others—despite multiple estimation outliers—generated no additional live flags, indicating that after the initial extreme events no further points surpassed the adaptive thresholds.

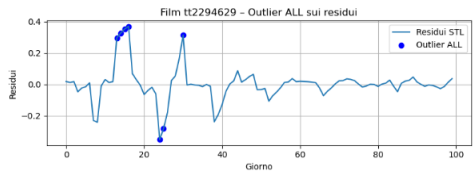


Figure 7.5: One of 5 series identified by all methods (estimation)

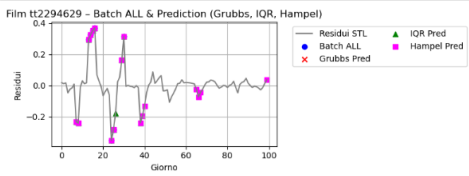


Figure 7.6: One of 5 series identified by all methods (prediction)

7.3.2 Anomaly Detection – Global

To identify films that exhibited a higher number of anomalies compared to the rest of the dataset over their full 100-day revenue trajectory, we employed two complementary approaches:

1. **Feature-based scoring.** Each film was represented using five numerical descriptors: total gross, maximum daily gross, standard deviation, number of local anomalies, and decay rate. These features were first transformed into standardized Z-scores, and then aggregated by summing the absolute values to obtain a global anomaly score. The films with the highest scores under this approach were `tt4154796` (Avengers: Endgame, 26.71), `tt2488496` (Star Wars: The Force Awakens, 22.78), `tt10872600` (Spider-Man: No Way Home, 20.75), `tt4154756` (Avengers: Infinity War, 18.33), and `tt2527336` (Star Wars: The Last Jedi, 17.86), indicating consistent extremity across several dimensions.

2. **Isolation Forest.** Using the same five features, we trained an Isolation Forest with a contamination rate of 1%. Anomaly scores were computed as the negative of the model's decision function, so that higher values indicated more isolated and anomalous points. The top-ranked films by this method were `tt4154796` (Avengers: Endgame, 0.0906), `tt2488496` (Star Wars: The Force Awakens, 0.0833), `tt10872600` (Spider-Man: No Way Home, 0.0714), `tt4154756` (Avengers: Infinity War, 0.0541), and `tt1825683` (Black Panther, 0.0541). Notably, the first four titles appear in both rankings, confirming a strong consistency between the two strategies.

To conclude our anomaly detection analysis, we will apply clustering techniques to verify and reinforce the previously identified anomalies. This step aims to uncover inherent groupings in the data and assess whether anomalous films tend to form distinct clusters, thus providing an additional layer of validation. Based on the insights gained, we will ultimately decide to retain the detected outliers, as they convey valuable information about extreme or atypical box office behaviors.

7.4 Motifs e Discords Analysis

The goal of this section is to identify recurring (*motifs*) and anomalous (*discords*) temporal patterns within the time series of movies, as a preparatory step for the classification phase. The analysis was carried out in multiple stages, moving from intra-series exploration (within each movie) to inter-series comparison (across different movies), and finally to an aggregation by rating category (see sec. 7.6.2).
Initially, we observed highly heterogeneous trends: some movies displayed decreasing patterns, while others showed periodic oscillations. To make the time series comparable and focus on the shape of the behavior rather than its scale, we applied logarithmic detrending, transforming each series into a sequence of smoothed logarithmic differences.
We tested different values for the window length m used in segmenting the series, evaluating the results both qualitatively and quantitatively. After several trials (including $m = 10, 20, 30$), we selected $m = 20$ as the optimal value, as it effectively captured significant patterns without being too local or too generic.

7.4.1 Intra-Series Analysis (Single Serie)

For each movie, we applied stumpy to compute the Matrix Profile on the detrended time series. From this, we extracted the main *motif* (the subsequence most similar to at least one other part of the same series) and the *discord* (the subsequence most dissimilar from all others). We then visualized the most representative cases.
Some movies exhibited highly repetitive motifs, while others showed unique peaks, which we interpreted as potential signs of sudden box office success or viral phenomena.

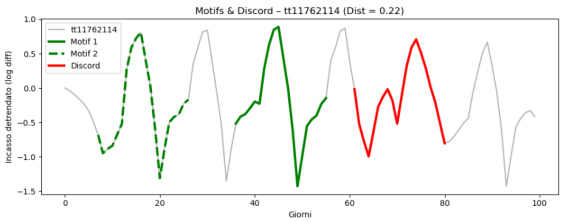


Figure A. The graph shows the detrended time series of movie `tt11762114`. Two segments are highlighted in green:
Motif 1 (solid line) and **Motif 2** (dashed line) are the most similar pair of subsequences in the series, with a very low Euclidean distance of 0.22. This indicates a highly repeated pattern with consistent behavior over time.
The red segment represents the **discord**, the most anomalous subsequence, i.e., the one most dissimilar from all others.
Interpretation: The low motif distance suggests that the movie exhibits recurring patterns over time, such as repeated cycles of revenue rises and drops. The discord, on the other hand, highlights a unique variation in the time series behavior, potentially linked to an isolated event (e.g., temporary withdrawal, re-release, or award).

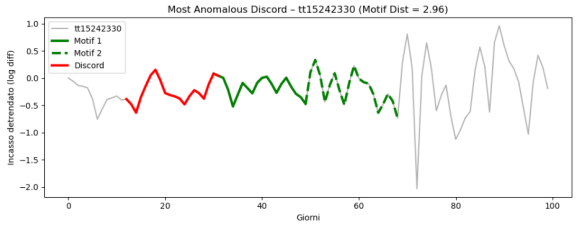


Figure B. This graph shows the detrended time series of movie `tt15242330`. The two most similar motifs (in green) have a Euclidean distance of 2.96, indicating weak similarity:
The shapes of **Motif 1** and **Motif 2** differ considerably, despite being the closest pair among all possible subsequences.
The **discord** (in red) appears in the first third of the series and shows a relatively stable pattern not replicated elsewhere.
Interpretation: This case represents a more irregular series, with no clear recurring patterns. The high motif distance suggests low repetitiveness in box office dynamics, while the discord still highlights a distinctive segment with behavior not seen in the rest of the series.

7.4.2 Inter-Series Analysis (Across Films)

In the second phase of the motif analysis, we extended the investigation to a global scale by comparing motifs across different films. We constructed a distance matrix based on the Euclidean distances between all motifs extracted in Phase 1 (intra-series), enabling the discovery of patterns that are recurrent or unique across the entire collection of time series.
From this matrix, we identified the Top-5 global motifs—that is, shape patterns that occurred in multiple films with minimal pairwise distance. These motifs represented highly shared temporal behaviors, such as gradual increases in revenue followed by soft declines, or symmetric bell-shaped curves centered around weekends or promotional events. Notably, while the absolute timing of these motifs varied from film to film, their normalized shape was consistent, revealing structural regularities common to mainstream cinematic releases.
Conversely, the Top-5 global discords represented motifs that were highly dissimilar from all others. These captured outlier behaviors, including sudden peaks (e.g., viral spikes), sharp drops (e.g., post-launch collapses), or erratic patterns indicative of unusual distribution strategies. Despite their brevity, these subsequences had a strong visual and structural impact on their respective time series, often deviating significantly from typical box office dynamics.
To further enrich the analysis, we computed motif-based distances between films, identifying the two most similar and two most dissimilar pairs. The most similar films shared both global motifs and overall trend structure, suggesting comparable marketing profiles or release contexts. In contrast, the most dissimilar films exhibited completely different patterns, confirming the presence of outliers or niche behavior in the dataset.
This global motif and discord analysis allowed us to move beyond local patterns and gain insights into recurring commercial dynamics across the cinematic landscape, as well as highlight singular cases of extreme success or failure.

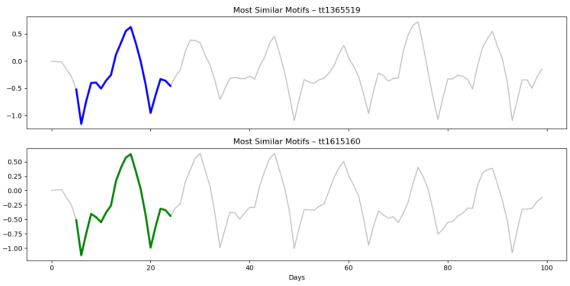


Figure A. Most similar motifs across two different films: `tt1365519` (*Tomb Raider*, 2018) and `tt1615160` (*The Foreigner*, 2017).

These motifs were identified through pairwise comparison of all subsequences from the dataset using normalized Euclidean distance. Both films belong to the action-thriller genre, have similar runtimes (around 1h55–2h), and exhibit a motif pattern with a steady rise followed by a sharp decline.

Despite differences in MPAA rating (PG-13 vs R) and total revenue ($58M vs $34M), the motifs suggest highly aligned revenue dynamics, likely reflecting a shared commercial trajectory driven by genre conventions and promotional strategies.

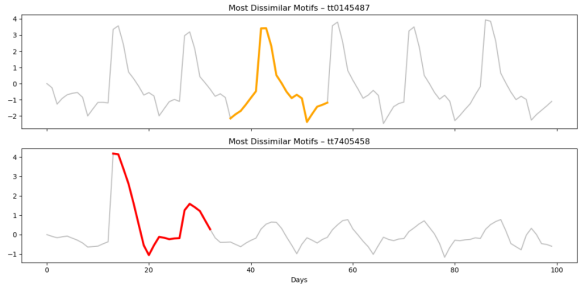


Figure B. Most dissimilar motifs between `tt0145487` (*Spider-Man*, 2002) and `tt7405458` (*A Man Called Otto*, 2022).

The motif from *Spider-Man* exhibits a sharp, early peak followed by rapid decay, typical of blockbuster openings fueled by hype. In contrast, the motif from *Otto* shows a gradual and sustained increase, likely associated with long-term reception and progressive release.

These motifs reflect opposite box office dynamics: fast initial success vs. slow audience buildup. Differences in genre (action sci-fi vs. comedy drama), audience, and gross revenue ($407M vs. $64M) support this divergence.

7.5 Clustering

In this section, we apply and compare two different clustering approaches on our time series dataset, with the goal of identifying meaningful groups of films based on their box office trajectories.

We explore two distinct perspectives: a shape-based approach using the KMeans algorithm, and a feature-based approach implemented through hierarchical clustering. The shape-based method operates directly on the normalized time series, allowing us to group films that exhibit similar temporal patterns. In contrast, the feature-based method involves transforming each series into a compact set of structural descriptors—such as total revenue, decay rate, and peak day—and performing clustering on these derived features.

Our objective is to compare the two methods not only quantitatively but also interpretatively, to better understand which films are grouped together and the criteria underlying their similarity.

7.5.1 Partitional Clustering: KMeans

To determine the optimal number of clusters k for KMeans clustering, we applied two techniques: the Elbow Method and the Silhouette Score, which evaluates the internal cohesion and separation between clusters. Both indicators suggested that $k = 5$ was a reasonable choice ($sil = 0.5213$), and we therefore adopted this value.

The application of the algorithm with $k = 5$ resulted in a highly unbalanced distribution of films across the clusters. Specifically, the largest cluster (3) contains 782 films, while the smallest (4, probably outliers) includes only 3. The remaining clusters contain C0: 253, C1: 71, and C2: 25 films respectively. This outcome indicates a heterogeneous behavior of the time series associated with each film.

Visual representations:

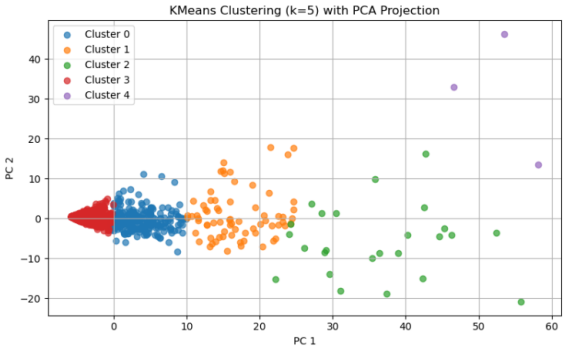


Figure 7.9: PCA Projection of KMeans Clusters

In the PCA plot, the clusters appear partially separated. Cluster 1 (71 films) and cluster 2 (25 films) are located in distinct areas of the space, suggesting that their temporal patterns differ significantly from the rest of the dataset. Cluster 0 and cluster 3, on the other hand, are positioned closer and show some overlap. However, their separation is supported by the average revenue curves (see Fig 7.11), which reveal distinct trends for the two groups.

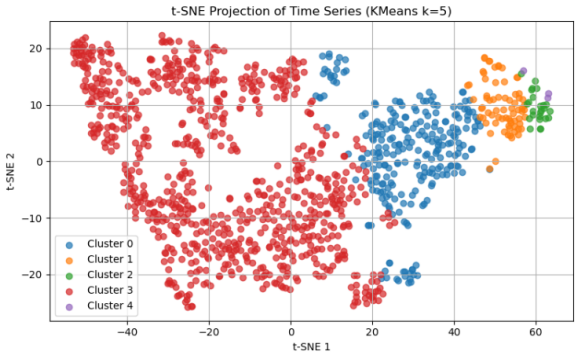


Figure 7.10: t-SNE Projection of KMeans Clusters

The t-SNE projection highlights the segmentation even more clearly. Cluster 3 forms a very compact region, confirming its internal consistency and low variance. Cluster 4, consisting of only 3 elements, is located in a peripheral and distant area, indicating clearly anomalous behavior. Cluster 2 shows a distinctive average curve shape, with a sharp initial peak and a steep decline, which is typical of explosively launched films.

Average Time Series, Distribution of Rating and Variance:

The average time series curves reveal clear distinctions among clusters. Cluster 3 (782 films) shows a smooth, flat pattern, typical of stable but modest box office trends, often associated with smaller productions or limited releases that maintain consistent, low-level performance over time. Cluster 2 features a sharp initial peak followed by a steep decline, indicating high initial hype without sustained success—frequently observed in blockbusters or sequels with strong openings but poor word-of-mouth. Cluster 4 displays multiple scattered peaks, suggesting staggered releases, irregular promotional efforts, or post-release events such as award nominations or streaming boosts.

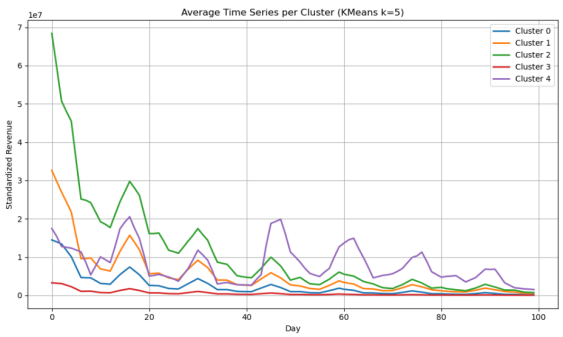


Figure 7.11: Average Time Series per Cluster

These dynamics align with intra-cluster variance: Cluster 3 is highly compact and homogeneous, whereas Clusters 2 and 4 show greater variability, reflecting more diverse and volatile revenue trajectories.

The distribution of average ratings across clusters highlights meaningful differences. Cluster 3 (782 films) is mainly composed of Medium and Medium Low ratings, reflecting its flat revenue patterns. Cluster 0 (253 films) and Cluster 1 (71 films) feature a higher concentration of well-rated films, with Cluster 1 being almost entirely High. Cluster 2 (25 films) is primarily linked to High and Medium High ratings, indicating films that are well-received or close to being highly appreciated. Cluster 4, while entirely 'High' rated, shows diverse time series behavior, pointing to variability in how its success was achieved.

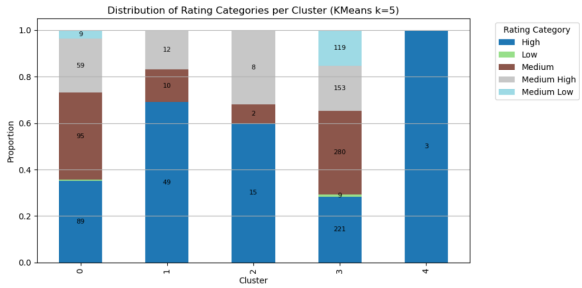


Figure 7.12: Average Rating per Cluster

7.5.2 Feature-based clustering.

To capture structural differences between films, a feature-based clustering was performed using five statistical descriptors (total gross,average gross, standard deviation, day of peak, decay rate). Unlike shape-based KMeans, this method summarizes each time series as a numerical vector. Agglomerative Clustering was applied with various linkage methods. Although $k = 2$ yielded the highest Silhouette Score (0.80), it was discarded due to poor interpretability. The configuration $k = 5$ (average linkage, Sil $= 0.4628$) was selected as a better balance. The resulting clusters were highly imbalanced: Cluster 2 dominated with 1,059 films, while clusters 3 and 4 are very small, with only 3 and 17 films respectively. Cluster 0 contains 43 films and cluster 1 includes 12. This distribution reflects the presence of one dominant group, along with a few minor and potentially more distinctive subgroups.

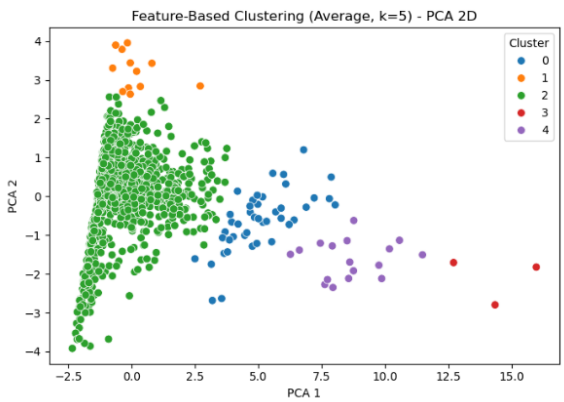


Figure 7.13: PCA Projection of Feature-Based Clusters

The PCA projection shows a reasonable separation between clusters. Cluster 2 (green) is the largest and most densely populated, confirming it represents the majority of the films. Clusters 0 (blue) and 4 (purple) are located more distinctly, while clusters 1 (orange) and 3 (red) are very small but clearly separated, suggesting well-defined and isolated behavioral patterns.

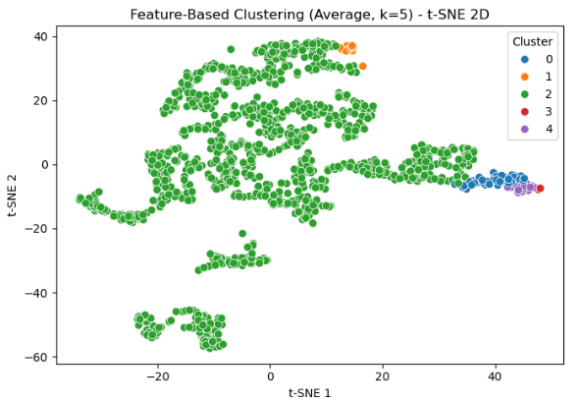


Figure 7.14: t-SNE Projection of Feature-Based Clusters

The t-SNE projection highlights the predominance of cluster 2, which spreads across most of the space with a fragmented but continuous distribution. The smaller clusters (0, 1, 3, and 4) are more compact and clearly separated on the right side. In particular, clusters 3 (red) and 4 (purple) are extremely isolated, confirming their uniqueness and anomalous nature within the dataset.

Average Times Series, distribution of rating and variance:

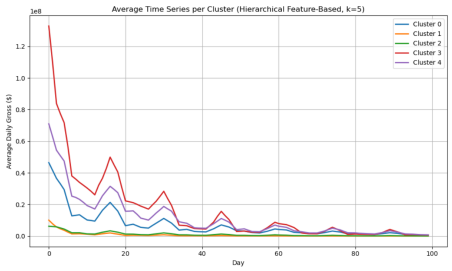


Figure 7.15: Average Time Series per Cluster

The plot of average time series per cluster highlights distinct behaviors: Cluster 3 (only 3 films) shows an early sharp peak with strong fluctuations, indicating sudden box office booms. Cluster 1 has a flat, low trend, likely reflecting low-performing films. Cluster 0 presents a more stable pattern with a mid-period peak. These dynamics are confirmed by intra-cluster variance, with Cluster 3 showing high variability and Cluster 1 appearing more compact.

Figure 7.16: Average Rating per Cluster

The stacked bar plot confirms earlier insights: Cluster 2 dominates in size and shows a balanced mix of Medium to High ratings. Cluster 1 contains mostly low-rated films, suggesting poor performance, while Cluster 3—though very small—includes only highly rated titles, indicating a distinct pattern. Clusters 0 and 4 fall in between in both size and rating diversity. These trends align with the intra-cluster variance analysis, especially for clusters 1 and 3.

7.5.3 Comparison and Conclusions

The comparison between KMeans and hierarchical clustering highlights two complementary approaches to film segmentation. KMeans, based on the shape of normalized time series, produced more balanced and interpretable clusters, effectively distinguishing between films with immediate success, regular performance, or atypical patterns. In contrast, hierarchical clustering, built on aggregated structural features, resulted in a more unbalanced structure with one dominant cluster and a few small but highly specific groups capable of isolating extreme cases in terms of performance or rating. The low Adjusted Rand Index (0.2084) confirms that the two methods yield different segmentations, each with distinct strengths: KMeans is better suited for general insights and predictive tasks, while hierarchical clustering excels at detecting outliers and homogeneous subgroups. Neither approach is inherently superior, but their combined use can provide a more comprehensive and layered representation of the dataset.

7.5.4 Cross-Analysis Between Clustering and Anomaly Detection

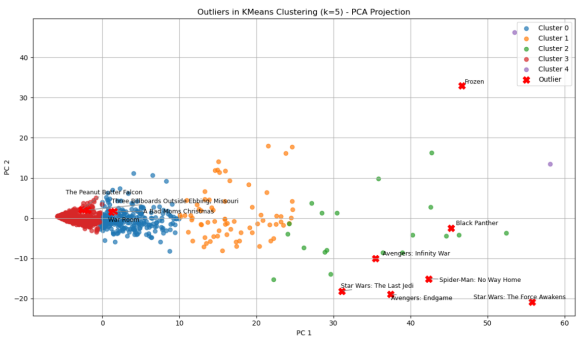


Figure 7.17: KMeans + outliers

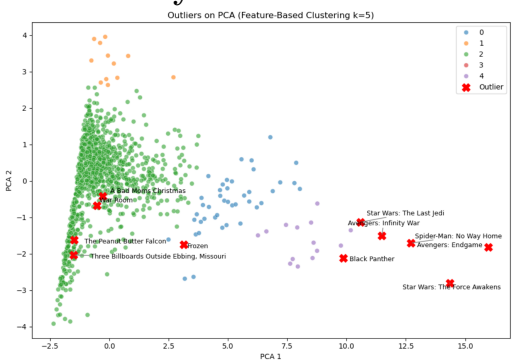


Figure 7.18: Hierarchical + outliers

A comparison was made between the results of anomaly detection(sec 7.3) and those of KMeans and hierarchical clustering, to evaluate their overlap in identifying outliers and their ability to

provide structural insight. KMeans assigned all global outliers to cluster 2, a group defined by very high gross, early peaks, low decay, and high ratings—typical blockbuster traits. Local outliers, however, were scattered across several clusters without clear separation, with some rare films like Frozen and Three Billboards appearing in smaller clusters due to a delayed revenue peak.
Hierarchical clustering showed stronger agreement with anomaly detection: all global outliers were isolated in marginal clusters, confirming their uniqueness, while local anomalies were absorbed into central groups. This reflects the fact that both global anomaly detection and hierarchical clustering rely on the same structural features, enhancing their alignment. In contrast, local detection methods, which focus on short-term irregularities, capture more subtle deviations. Overall, the methods are complementary, offering a broader understanding of anomalous film behaviors.

7.6 Classification

We addressed a time series classification task with the goal of predicting the `rating_category` of each film. The target variable includes five ordinal classes and reflects the overall audience evaluation.

7.6.1 K-Nearest Neighbors (KNN)

KNN with Euclidean and/or Manhattan We assessed the performance of the K-Nearest Neighbors (KNN) classifier on the time series dataset by exploring various configurations and addressing class imbalance. The initial evaluation, using Euclidean distance and varying values of k, showed that performance improved with larger k, peaking at 49% accuracy and a macro F1 score of 0.45 for $k = 17$. However, the model consistently failed to correctly classify minority classes, particularly the `Low` category.
To improve performance, we applied grid search with stratified train-test splitting, testing both Euclidean and Manhattan distances along with weighting strategies (`uniform`, `distance`). The best configuration—$k = 19$ with Manhattan distance and distance-based weights—achieved 53% accuracy and a macro F1 score of 0.345. Still, recognition of underrepresented classes remained weak.
We then experimented with balancing techniques such as ENN and SMOTE, but these offered limited improvements. The most effective strategy proved to be merging the `Low` and `Medium Low` categories into a single `Low-Mid` class, thereby improving class distribution. With this new target variable, the best model—$k = 7$, Euclidean distance, and uniform weights—achieved 48.7% accuracy and a macro F1 score of 0.412, resulting in a more balanced classification across all categories.

Class	Precision	Recall	F1
High	0.53	0.67	0.59
Low-Mid	0.60	0.21	0.32
Medium	0.47	0.60	0.53
Medium High	0.32	0.16	0.21
Accuracy	0.487 (341 samples)		
Macro avg	0.48	0.41	0.41
Weighted avg	0.50	0.49	0.46

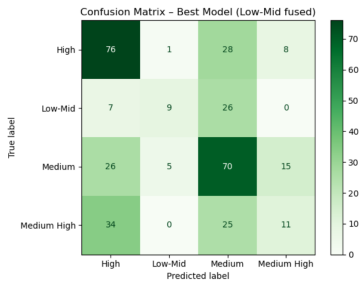


Figure 7.19: Confusion Matrix Euclidean

KNN with Dynamic Time Warping (DTW) We evaluated KNN classification using Dynamic Time Warping (DTW) on a stratified subset of 300 films, initially without a warping window and using uniform weights. Performance improved up to $k = 5$, stabilized between

$k = 9$ and $k = 13$, and slightly declined at $k = 17$, leading us to select $k = 13$ as the optimal value. On the full dataset, with $k = 13$ and `weights='distance'`, the model achieved an accuracy of 43%, a macro F1-score of 0.29, and a weighted F1-score of 0.40. While the `Medium` class was well recognized (F1 = 0.59), minority classes such as `Low` and `Medium High` showed poor performance.
To address class imbalance, we merged `Low` and `Medium Low` into a `Low-Mid` class and repeated tuning on a new 300-film subset. The best setting $k = 9$, `weights='uniform'` reached a macro F1 of 0.41. Applied to the full dataset, this setup maintained 43% accuracy, improved macro F1 to 0.35, and preserved weighted F1 at 0.40. `Medium` (F1 = 0.49) and `High` (F1 = 0.53) were well classified, but `Low-Mid` (F1 = 0.28) and `Medium High` (F1 = 0.12) remained problematic due to frequent confusion with `Medium`.

Class	Precision	Recall	F1
High	0.51	0.54	0.53
Low-Mid	0.41	0.21	0.28
Medium	0.41	0.60	0.49
Medium High	0.21	0.09	0.12
Accuracy	0.43 (341 samples)		
Macro avg	0.39	0.36	0.35
Weighted avg	0.42	0.43	0.40

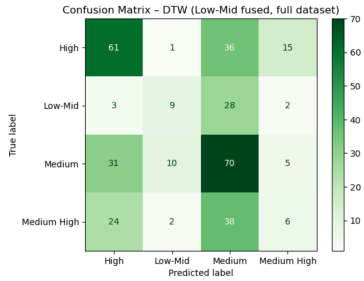


Figure 7.20: Confusion Matrix – DTW

Conclusion: The best model Comparing the best configurations obtained with the two distance metrics, it is clear that the KNN model with Euclidean distance and k=7, applied to the dataset with the Low-Mid class fusion, represents the most balanced solution. This model achieved an accuracy of 48.7%, a macro F1-score of 0.412, and a weighted F1-score of 0.459. In contrast, the best DTW-based model—despite benefiting from class fusion and parameter tuning—reached only a macro F1-score of 0.35 and an overall accuracy of 43%. Moreover, DTW required significantly more computational time without offering a clear advantage in predictive performance.
In conclusion, despite its greater flexibility in capturing temporal similarity, DTW did not outperform the simpler Euclidean distance, which proved more effective in distinguishing box office patterns associated with different rating categories.

7.6.2 Shapelets

We extended the rating classification by applying a shapelet-based approach. After converting the time series into a 3D format and encoding the target labels, we used a Gradient-based shapelet learning approach, implemented through the ShapeletModel class from the tslearn library to extract 30 shapelets of lengths 10, 20, and 30. The model was trained using the Adam optimizer with 800 epochs, L2 regularization, and a batch size of 256. Once the extraction was complete, each time series was represented by its minimum distances to the learned shapelets, and the classifier was tested on this new feature space. The best performance was achieved by a K-Nearest Neighbors classifier (see Fig 7.22)(with $p = 1$, 13 neighbors, and `weights='distance'`), reaching 42% accuracy and a macro F1-score of 0.33. The confusion matrix shows that the *High* and *Medium* classes were the most accurately predicted, while *Low-Mid* and *Medium High* were frequently misclassified, indicating overlapping temporal dynamics between adjacent rating levels.

Table 7.3: Metrics for shapelet-based KNN model

Class	Precision	Recall	F1-score	Support
High	0.45	0.58	0.50	113
Low-Mid	0.38	0.12	0.18	42
Medium	0.45	0.54	0.49	116
Medium High	0.22	0.13	0.16	70
Accuracy		0.42		341
Macro avg	0.37	0.34	0.33	341
Weighted avg	0.39	0.42	0.39	341

Shapelets compare to motifs: For each rating class, the top 10 motifs and top 10 discords were identified using the Matrix Profile, allowing us to highlight distinctive patterns in the time series. High-rated films exhibit regular dynamics, typically with a well-defined initial peak followed by a consolidation phase, while lower-rated titles show irregular and unstable sequences with isolated peaks. All motifs and discords were stored along with their metadata: rating category, type (motif or discord), starting index, distance to the nearest neighbor, and full sequence of values.

To assess the quality of the shapelets learned by the model, we performed a direct comparison with the motifs previously extracted for the same rating categories, using the Dynamic Time Warping (DTW) distance. Prior to the computation, both shapelets and motifs were normalized using Z-score to ensure scale homogeneity and reliable similarity measurement. The comparison was carried out consistently, only between shapelets and motifs belonging to the same class (e.g., *Medium* or *High*), and in many cases revealed low DTW distances (ranging from 0.7 to 1.2). This suggests that the model effectively captured meaningful subsequences that align well with the typical temporal structures identified by unsupervised motif discovery, confirming the effectiveness of supervised learning in capturing recurring patterns. However, predictive performance remained limited, especially in distinguishing adjacent or underrepresented classes. As such, the strength of this approach lies more in its interpretability than in its classification accuracy.
We complemented the shapelet-based approach with a semi-supervised classification strategy using motifs and discords, employing both K-Nearest Neighbors and Decision Tree models. The results were very similar to those obtained with shapelets, with an accuracy of 0.33 for KNN and 0.31 for Decision Tree. Other metrics and the confusion matrices showed comparable patterns, highlighting similar difficulties in recognizing underrepresented classes and confirming that both methods capture relevant temporal patterns, though with room for improvement.

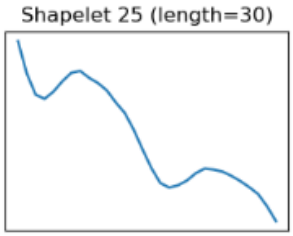


Figure 7.21: Example of extracted shapelet

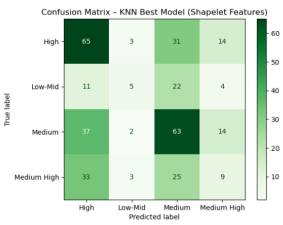


Figure 7.22: Shapelet-based results

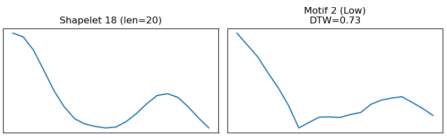


Figure 7.23: Shapelet vs Motif

7.6.3 Deep Classification

ROCKET: We employed the ROCKET (RandOm Convolutional KErnel Transform) method to transform time series into high-dimensional feature vectors via 10,000 random convolutional

kernels, capturing multiscale temporal patterns. A RidgeClassifier was then trained on these features, offering an efficient and effective method for time series classification.
The classification results are shown below. This outperformed previous models in terms of overall accuracy and macro F1-score, especially improving recognition of the *Medium* class. However, distinguishing less represented classes like *Low-Mid* and *Medium High* remains challenging.

Class	**Precision**	**Recall**	**F1**	**Support**
High	0.57	0.56	0.57	113
Low-Mid	0.46	0.26	0.33	42
Medium	0.48	0.66	0.55	116
Medium High	0.31	0.21	0.25	70
Accuracy	0.48 (341 samples)			
Macro avg	0.46	0.42	0.43	341
Weighted avg	0.47	0.48	0.47	341

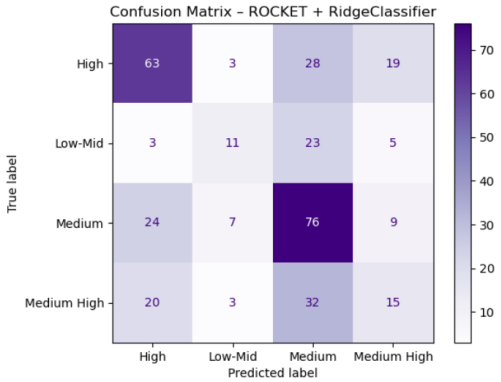


Figure 7.24: Confusion Matrix ROCKET

CNN: We trained a 1D Convolutional Neural Network (CNN) on normalized time series data to classify films by rating. Despite using a deep architecture with regularization and early stopping, the model achieved low performance: 26% accuracy and macro F1-score of 0.23. The confusion matrix shows that most predictions collapsed onto the "Low-Mid" class, with high recall (79%) but poor precision and weak recognition of other classes. This suggests that the CNN struggled due to class imbalance and overlapping patterns.

Class	**Precision**	**Recall**	**F1**	**Support**
High	0.76	0.12	0.20	113
Low-Mid	0.17	0.79	0.27	42
Medium	0.34	0.33	0.33	116
Medium High	0.36	0.07	0.12	70
Accuracy	0.26 (341 samples)			
Macro avg	0.41	0.32	0.23	341
Weighted avg	0.46	0.26	0.24	341

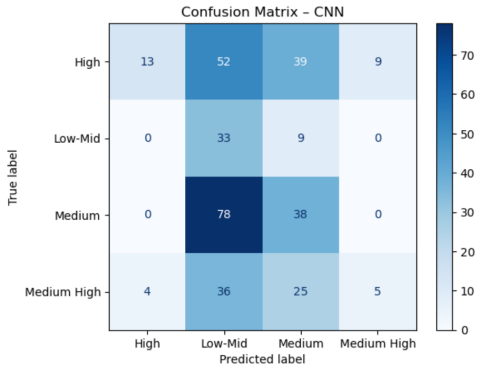


Figure 7.25: Confusion Matrix CNN

7.6.4 Conclusion

Model	**Precision**	**Recall**	**F1-score**	**Accuracy**
KNN (Euclidean)	0.48	0.41	0.41	0.487
KNN (DTW)	0.39	0.36	0.35	0.43
Shapelet-based	0.37	0.34	0.33	0.42
ROCKET	0.46	0.42	0.43	0.48
CNN	0.41	0.32	0.23	0.26

Table 7.4: Comparison of Classification Models on Time Series

The comparative analysis of the models shows that ROCKET with RidgeClassifier was the most effective approach among those tested, achieving the highest macro F1-score (0.43) and an accu-

racy of 48%, with a well-balanced performance across all classes, including the underrepresented ones. This result is particularly noteworthy given that ROCKET does not rely on supervised transformations, yet it successfully captures meaningful local patterns through the use of 10,000 random convolutional kernels.
The KNN model with Euclidean distance also performed well (macro F1 = 0.41, accuracy = 48.7%), especially on the High and Medium classes, although it was slightly less balanced overall compared to ROCKET.
In contrast, the shapelet-based and KNN with DTW models proved less effective, with macro F1-scores of 0.33 and 0.35 respectively. These models struggled to distinguish between adjacent or less frequent classes, despite the interpretability advantage offered by shapelets.
Lastly, the CNN, although a more complex model, yielded the poorest results (macro F1 = 0.23, accuracy = 26%), showing signs of overfitting and substantial class confusion. This is likely due to data scarcity and the lack of a network architecture optimized for the specific characteristics of this dataset.
In summary, ROCKET offers the best trade-off between accuracy, class balance, and robustness, making it the most suitable model for time series classification in our dataset.

7.7 Sequential Pattern Mining

To analyze temporal trends in box office performance, we applied Sequential Pattern Mining techniques on the time series of daily revenues. After discretizing each series using the SAX algorithm (with n_bins = 6 and 7), we converted numeric values into symbolic levels from `a` (very low) to `f/g` (very high), preserving repeated symbols to retain temporal structure.
We initially used Apriori, treating each film as an unordered transaction of symbols, but this approach yielded trivial itemsets with full support due to excessive symbol overlap. To better capture sequential dynamics, we adopted PrefixSpan, which identifies frequent ordered subsequences.
On the full dataset, dominant patterns such as `['f', 'f', 'f']` and `['f', 'f', 'f', 'f', 'e']` emerged, indicating peak revenues followed by a gradual decline. However, these patterns appeared too frequently across all films to effectively distinguish between different types. To uncover more meaningful trends, we applied PrefixSpan within time series clusters and across `rating_category` classes.
Clusters 0 and 3 showed persistent high-income trends, while Cluster 2 revealed rapid declines from high to low values. Cluster 1 lacked dominant sequences, indicating internal variability. Similarly, High-rated films displayed stable patterns with symbols `f` and `e`, while Low-rated ones transitioned quickly to lower levels (`a-d`), consistent with poor or short-lived box office performance.
We summarized these findings using heatmaps of pattern support and symbolic visualizations, which highlighted key differences between groups. Sequential pattern mining thus proved valuable in capturing behavioral trends that complement clustering and classification analyses.

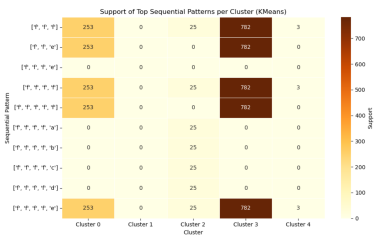


Figure 7.26: Heatmap for support (KMeans)

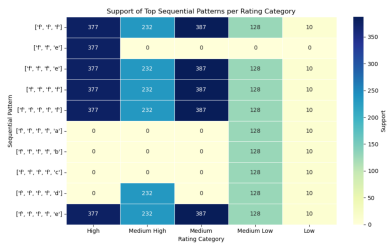


Figure 7.27: Heatmap for support (rating_category)